

Rapport de sécurité — Audit de l'application OWASP Juice Shop

Auteur : Ibrahima Lo — Licence 3, Cybersécurité

Rôle : Cybersecurity Analyst / Junior DevSecOps Engineer

Application cible : OWASP Juice Shop (clone local patché), déployée via Docker Desktop

Environnement : machine physique locale + Docker Desktop (Windows, WSL2)

Sommaire

1. Introduction
 2. Méthodologie
 3. Vulnérabilités identifiées
 4. Classification CWE
 5. Analyse des impacts (CIA)
 6. Remédiations
 7. Pipeline de sécurité
 8. Résultats
 9. Décision de déploiement
 10. Conclusion
- Annexe — Partie 6 : Analyse critique

1. Introduction

Ce rapport présente l'évaluation de sécurité réalisée sur une application web de démonstration (OWASP Juice Shop) avant sa mise en production simulée. L'application permet la création de compte, l'authentification et l'accès à des ressources (produits, paniers, commandes, profils) contenant des données utilisateur potentiellement sensibles.

L'objectif de cette mission est d'identifier les vulnérabilités susceptibles de compromettre la confidentialité, l'intégrité ou la disponibilité de ces données, d'en évaluer la criticité, de proposer et d'implémenter des remédiations, puis d'intégrer des contrôles de sécurité automatisés dans un pipeline CI/CD (Jenkins) afin d'empêcher la régression de ces problèmes.

2. Méthodologie

L'analyse combine :

- **Revue de code manuelle (SAST manuel)** sur les composants critiques : authentification (routes/login.ts), gestion du panier (routes/basket.ts), module de sécurité (lib/insecurity.ts), et frontend Angular (search-result.component.ts).
- **Analyse statique outillée (SAST)** : Semgrep, règles p/owasp-top-ten, p/javascript, p/typescript.
- **Analyse des dépendances (SCA)** : Trivy (scan filesystem des package.json / package-lock.json).
- **Détection de secrets** : Gitleaks sur l'ensemble du code source.
- **Analyse dynamique (DAST)** : OWASP ZAP Baseline Scan contre l'application démarrée dans un conteneur Docker.
- **Analyse d'impact** : classification CWE de chaque vulnérabilité, puis évaluation des trois propriétés CIA (Confidentialité / Intégrité / Disponibilité).

Toutes les analyses sont automatisées et rejouables via le pipeline Jenkins décrit en section 7, ce qui garantit la reproductibilité des résultats.

Les captures d'écran servant de preuve (exploitation live de chaque vulnérabilité, résultats bruts des outils, vues du pipeline Jenkins) sont fournies séparément dans le dossier screenshots/ du dépôt Git, plutôt qu'intégrées dans ce document.

3. Vulnérabilités identifiées

ID	Vulnérabilité	Composant	CWE	Description
V1	SQL Injection (login bypass)	routes/login.ts:34	CWE-89	La requête d'authentification concatène email et password directement dans un template literal SQL exécuté par sequelize.query. Un payload tel que ' OR 1=1 - - permet de s'authentifier sans connaître le mot de passe.
V2	Cross-Site Scripting (DOM-based)	frontend/.../search-result.component.ts:144	CWE-79	Le paramètre de recherche q (contrôlé par l'utilisateur via l'URL) est marqué comme "sûr" via sanitizer.bypassSecurityTrustHtml(queryParam) puis injecté via [innerHTML], contournant la protection anti-XSS d'Angular.
V3	Broken Access Control / IDOR	routes/basket.ts:15-31	CWE-639	GET /rest/basket/:id renvoie le panier correspondant à l'id fourni dans l'URL sans vérifier qu'il appartient à l'utilisateur authentifié — un attaquant peut lire (et, via d'autres routes du même module, modifier) le panier de n'importe quel autre utilisateur en changeant l'ID.
V4	Secret codé en dur (clé privée JWT)	lib/insecurity.ts:21	CWE-798	La clé privée RSA utilisée pour signer tous les tokens d'authentification (RS256) est écrite en clair dans le code source versionné, permettant à quiconque a accès au dépôt de forger des tokens valides (y compris administrateur).
V5	Hachage de mot de passe faible	lib/insecurity.ts:41	CWE-916	Les mots de passe sont hachés avec MD5 sans sel (crypto.createHash('md5')), un algorithme rapide et cassé, facilitant les attaques par force brute / rainbow tables en cas de fuite de la base.

ID	Vulnérabilité	Composant	CWE	Description
V6	Dépendances vulnérables / obsolètes	package . json	CWE-1104	Le projet dépend de versions très anciennes de bibliothèques de sécurité : jsonwebtoken@0.4.0, express-jwt@0.1.3, sanitize-html@1.4.2, exposées à des CVE connues (bypass de vérification de signature, contournement de sanitization HTML).

4. Classification CWE

CWE	Nom	Vulnérabilité(s) concernée(s)
CWE-89	SQL Injection	V1
CWE-79	Cross-Site Scripting	V2
CWE-639	Authorization Bypass Through User-Controlled Key (IDOR)	V3
CWE-798	Use of Hard-coded Credentials	V4
CWE-916	Use of Password Hash With Insufficient Computational Effort	V5
CWE-1104	Use of Unmaintained Third-Party Components	V6

5. Analyse des impacts (CIA)

Vulnérabilité	Confidentialité	Intégrité	Disponibilité	Criticité	Priorité
V1 SQL Injection	Oui — contournement d'authentification, accès à tout compte	Oui — requêtes additionnelles possibles selon le point d'injection	Possible — requête malformée pouvant dégrader le service	Critical	1 (immédiate)
V2 XSS (DOM)	Oui — vol de session/token via script injecté	Possible — manipulation du DOM, phishing in-page	Non	High	2
V3 IDOR (panier)	Oui — lecture des paniers d'autres utilisateurs	Possible — si les routes de modification partagent la même faille	Non	Critical	1 (immédiate)
V4 Secret JWT codé en dur	Oui — forge de tokens, usurpation de n'importe quel compte	Oui — un token forgé "admin" permet de modifier toutes les données	Possible — un compte admin forgé peut supprimer des ressources	Critical	1 (immédiate)
V5 Hash MD5 mots de passe	Oui — cassage rapide des mots de passe en cas de fuite	Non	Non	Medium	3

Vulnérabilité	Confidentialité	Intégrité	Disponibilité	Criticité	Priorité
V6 Dépendances vulnérables	Possible — selon CVE (ex. bypass de vérif. JWT)	Possible	Possible (CVE de type ReDoS)	High	2

Justification de priorisation : V1, V3 et V4 sont classées critiques car chacune permet, individuellement, une compromission totale de la confidentialité des données (et souvent de l'intégrité) sans nécessiter de privilège préalable ; elles doivent bloquer la mise en production tant qu'elles ne sont pas corrigées. V2 et V6 sont élevées mais nécessitent soit une interaction utilisateur (XSS), soit l'exploitation d'une CVE tierce spécifique. V5 est importante mais n'est exploitable qu'en cas de fuite préalable de la base de données (impact différé).

6. Remédiations

Voir le détail complet (cause / correction / justification / vérification) dans `remediation/README.md` (`../remediation/README.md`). Résumé :

Vulnérabilité	Cause	Remédiation	Vérification
V1 SQL Injection	Entrées non paramétrées (concaténation SQL)	Requêtes paramétrées (replacements Sequelize)	Test manuel (payload rejeté) + re-scan Semgrep (0 finding)
V3 IDOR (panier)	Absence de vérification de propriété sur l'ID de ressource	Contrôle serveur : bid du token doit correspondre à l'ID demandé, sinon 403	Test manuel (accès croisé refusé) + re-scan DAST ZAP
V4 Secret JWT codé en dur	Clé privée en clair dans le code versionné	Rotation de la clé + chargement depuis un fichier exclu du dépôt (jwt.key, .gitignore)	Re-scan Gitleaks (0 secret détecté) + invalidation des anciens tokens

Les vulnérabilités V2, V5 et V6 sont documentées avec une remédiation proposée (non implémentée dans ce dépôt de démonstration) : encodage de sortie contextuel au lieu de `bypassSecurityTrustHtml` (V2), migration vers `bcrypt/argon2` avec sel (V5), mise à jour des versions de dépendances (`npm audit fix / upgrade majeur`) (V6).

7. Pipeline de sécurité

Un pipeline Jenkins (`Jenkinsfile`, à la racine du dépôt) automatise les contrôles de sécurité à chaque modification du code :

1. Checkout
2. Build / Preparation (construction de l'image Docker de l'app cible + démarrage)
3. Security Analysis (séquentiel : SAST Semgrep · SCA Trivy · Secret Detection Gitleaks)
4. Additional Security Check (DAST OWASP ZAP baseline sur l'app en cours d'exécution)
5. Report Generation (agrégation des résultats -> `reports/summary.md`)
6. Notification (résumé + décision automatique de seuil)

Les trois scans du stage 3 s'exécutent séquentiellement plutôt qu'en parallèle : sur la

machine de laboratoire utilisée (8 Go de RAM), les lancer simultanément provoquait des plantages du moteur Docker Desktop par manque de mémoire.

Outil	Type	Vulnérabilités détectables	Limites	Étape du pipeline
Semgrep	SAST	Injections (SQL/commande), usage dangereux d'API (innerHTML, eval), mauvaises pratiques de crypto	Ne détecte que les patterns couverts par ses règles ; faux négatifs sur logique métier complexe ; pas d'exécution réelle du code	Stage 3 (1er des 3 scans)
Trivy	SCA	CVE connues dans les dépendances npm (versions vulnérables)	Ne détecte que les CVE déjà publiées/répertoriées ; ne dit rien sur le code métier propre	Stage 3 (2e des 3 scans)
Gitleaks	Secret detection	Clés API, tokens, clés privées, mots de passe codés en dur	Basé sur des expressions régulières / entropie : faux positifs (données aléatoires légitimes) et faux négatifs (secrets obfusqués/encodés) possibles	Stage 3 (3e des 3 scans)
OWASP ZAP (baseline)	DAST	XSS reflété, en-têtes de sécurité manquants, configuration TLS/cookies, certaines injections détectables dynamiquement	Ne couvre que ce qu'il peut atteindre par crawl automatique (pas d'authentification avancée par défaut) ; ne détecte pas l'IDOR sans scénario dédié ; scan "baseline" volontairement non intrusif	Stage 4, après démarrage de l'application

8. Résultats

Le pipeline Jenkins a été exécuté deux fois : une première fois sur le code d'origine (vulnérable), une seconde fois après application des trois correctifs de la section 6. Les rapports bruts de chaque outil sont archivés dans `reports/` (`semgrep-report.json`, `trivy-report.json`, `gitleaks-report.json`, `zap-report.json/html/md`, `summary.md`).

Outil	Avant remédiation	Après remédiation	Interprétation
Semgrep (SAST)	9 findings (3 error / 6 warning)	7 findings (2 error / 5 warning)	-2 findings, cohérent avec la correction de l'injection SQL (V1)
Trivy (SCA)	8 critical / 45 high / 33 medium / 6 low	8 critical / 45 high / 33 medium / 6 low (inchangé)	attendu : V6 (dépendances) a été documentée mais pas corrigée dans ce dépôt de démonstration
Gitleaks (secrets)	67 secrets détectés	66 secrets détectés	-1, exactement la clé privée RSA retirée de <code>lib/insecurity.ts</code> (V4)

Outil	Avant remédiation	Après remédiation	Interprétation
OWASP ZAP (DAST)	échec technique du premier run (corrigé, voir ci-dessous)	Medium : 2, Low : 5, Informational : 3	premier scan DAST complet et exploitable

Note méthodologique : le tout premier run du pipeline n'a produit aucun résultat DAST — le script `zap-baseline.py` refuse de s'exécuter si son répertoire de travail (`/zap/work`) n'est pas un point de montage réel, ce qui n'était pas le cas dans une première version du `Jenkinsfile`. Corrigé en montant un volume Docker nommé dédié avant le second run.

Findings notables du scan ZAP (mauvaise configuration de sécurité, catégorie complémentaire aux 6 vulnérabilités de la section 3) :

- Content Security Policy (CSP) Header Not Set
- Cross-Domain Misconfiguration
- Cross-Origin-Embedder-Policy Header Missing or Invalid
- Dangerous JS Functions (fonctions JavaScript sensibles exposées côté client)

Ces alertes sont de sévérité Medium/Low et n'ont pas été retenues comme vulnérabilité principale (V1-V6) mais confirment la valeur du DAST en complément de l'analyse manuelle et du SAST : elles portent sur la configuration HTTP de l'application plutôt que sur son code métier.

Exemple de classification par priorité (issue de PARTIE 5 de l'énoncé) :

Problème	CWE	Sévérité	Action recommandée
SQL Injection (V1)	CWE-89	Critical	Bloquer le déploiement
IDOR panier (V3)	CWE-639	Critical	Bloquer le déploiement
Secret JWT codé en dur (V4)	CWE-798	Critical	Bloquer le déploiement
XSS (V2)	CWE-79	High	Correction requise avant déploiement
Dépendances vulnérables (V6)	CWE-1104	High	Correction requise avant déploiement
Hash MD5 (V5)	CWE-916	Medium	Correction planifiée (sprint suivant)

9. Décision de déploiement

[ROUGE] Reject Deployment (avant remédiation) / **[ORANGE] Accept with Conditions** (après application des 3 correctifs critiques V1/V3/V4, sous réserve de planifier V2/V5/V6).

Justification : trois vulnérabilités critiques (V1, V3, V4) permettaient, avant correction, une compromission totale de la confidentialité et de l'intégrité des données de tous les utilisateurs sans nécessiter de privilège préalable — un déploiement en l'état aurait exposé l'organisation à une fuite de données massive et à une usurpation de compte

administrateur. Après application des correctifs (section 6) et vérification par re-scan des outils, ces trois risques critiques sont neutralisés. Les vulnérabilités restantes (V2, V5, V6) sont d'un impact plus limité ou nécessitent une interaction/condition supplémentaire ; elles sont acceptables temporairement à condition d'être planifiées et corrigées dans un délai court (prochain sprint), avec un contrôle de non-régression assuré par le pipeline Jenkins à chaque nouvelle modification du code.

10. Conclusion

Cet audit a permis d'identifier six vulnérabilités couvrant plusieurs catégories de l'OWASP Top 10 (injection, XSS, contrôle d'accès défaillant, exposition de secrets, cryptographie faible, composants vulnérables), de les analyser selon les trois propriétés fondamentales de la sécurité des données, et de corriger les trois vulnérabilités les plus critiques avec vérification effective de leur correction. L'intégration de ces contrôles (SAST, SCA, secret detection, DAST) dans un pipeline Jenkins automatisé permet de transformer un audit ponctuel en un garde-fou continu, capable de détecter une régression avant toute mise en production future. Comme discuté en Partie 6 (analyse critique), ces outils automatisés réduisent fortement le risque mais ne remplacent pas l'analyse humaine, notamment pour les failles de logique métier (comme l'IDOR V3) qu'aucun scanner générique ne peut garantir de détecter à 100 %.

L'analyse critique complète (Partie 6 de l'énoncé) est présentée en annexe de ce document.

Annexe — Partie 6 : Analyse critique

****Question :** Un outil de sécurité qui ne détecte aucune vulnérabilité peut-il garantir qu'une application est sécurisée ?**

Non. L'absence de finding remonté par un outil signifie seulement que ****cet outil, avec sa configuration actuelle, n'a rien trouvé**** — ce n'est en aucun cas une preuve d'absence de vulnérabilité. Cette limite se comprend à travers plusieurs angles complémentaires.

Faux négatifs. Chaque outil ne détecte que ce que ses règles ou son moteur savent reconnaître. Sengrep ne signale que les patterns couverts par son jeu de règles ; il n'aurait par exemple jamais détecté seul la faille IDOR sur `/rest/basket/:id` de ce rapport (V3), car il s'agit d'un défaut de **logique métier** — le code est syntaxiquement correct, il manque simplement une vérification d'autorisation que l'outil ne peut pas deviner sans connaître les règles métier de l'application. De même, Trivy et Gitleaks ne trouvent que des CVE publiées ou des motifs de secrets connus : un secret encodé différemment, ou une dépendance vulnérable non encore répertoriée dans une base CVE, passera inaperçu. OWASP ZAP en mode « baseline » est volontairement non intrusif et ne couvre que ce que son crawler automatique atteint, sans authentification avancée ni scénarios métier complexes.

Faux positifs. À l'inverse, un outil « silencieux » peut aussi l'être parce qu'il a été mal configuré (règles désactivées, chemin exclu, seuil de sévérité trop élevé) plutôt que parce que l'application est saine — et un outil trop permissif en configuration par défaut peut masquer de vrais problèmes tout en générant, sur d'autres projets, des alertes non pertinentes qui, à force, entraînent une désensibilisation de l'équipe (« alert fatigue »), un risque tout aussi dangereux qu'un faux négatif.

Couverture des règles. Un SAST/DAST/SCA couvre un ensemble de catégories connues (souvent alignées sur l'OWASP Top 10 ou des bases CWE), mais la surface de risque réelle d'une application (architecture, flux de données propriétaires, règles métier de contrôle d'accès, interactions entre composants) dépasse toujours ce périmètre. Dans ce projet, sur les six vulnérabilités identifiées, seule une partie (SQLi, secrets, dépendances) est directement du ressort d'un outil automatisé ; l>IDOR et certains choix d'architecture nécessitent une revue humaine ciblée.

Importance de l'analyse humaine. Un pipeline automatisé est un filet de sécurité nécessaire — il garantit la reproductibilité et empêche la régression des problèmes déjà connus — mais il doit être complété par une revue de code manuelle, une analyse de risques

contextuelle et, idéalement, un test d'intrusion ciblé sur la logique métier. La bonne pratique n'est donc jamais « le scanner est vert, on déploie », mais « le scanner ne remonte rien connu, **et** un humain a validé les zones à risque spécifiques à l'application » avant toute décision de mise en production.

(~ 420 mots)